

Module: Optimization – Gradient Descent & Optimization in Practice

04 July 2026 10:00

Core Idea of the Class:

*A model learns by making mistakes,
measuring those mistakes,
and correcting itself step by step.*

Class Roadmap

30 June 2026 20:08

By the end of this session, you should understand:

1. Why models need optimization
2. What loss means
3. What gradient descent does
4. Why learning rate matters
5. Difference between Batch GD, SGD and Mini-Batch GD
6. How this applies to ML and Deep Learning

Why Optimization Matters

30 June 2026 20:09

Why do we even need optimization?

First attempt are rarely perfect



Cooking



Exam prep



Driving

First attempt are rarely perfect.

We try → We observe result → We adjust

How do we know what to change?

FEEDBACK

GOAL

What does optimization mean?

Optimization -->

means choosing the better actions based on feedback to achieve a goal



Too slow

Middle

Too fast



Real-Life Intuition

30 June 2026 20:09

Optimization is not blindly increasing something.

It is adjusting step by step toward the best result.

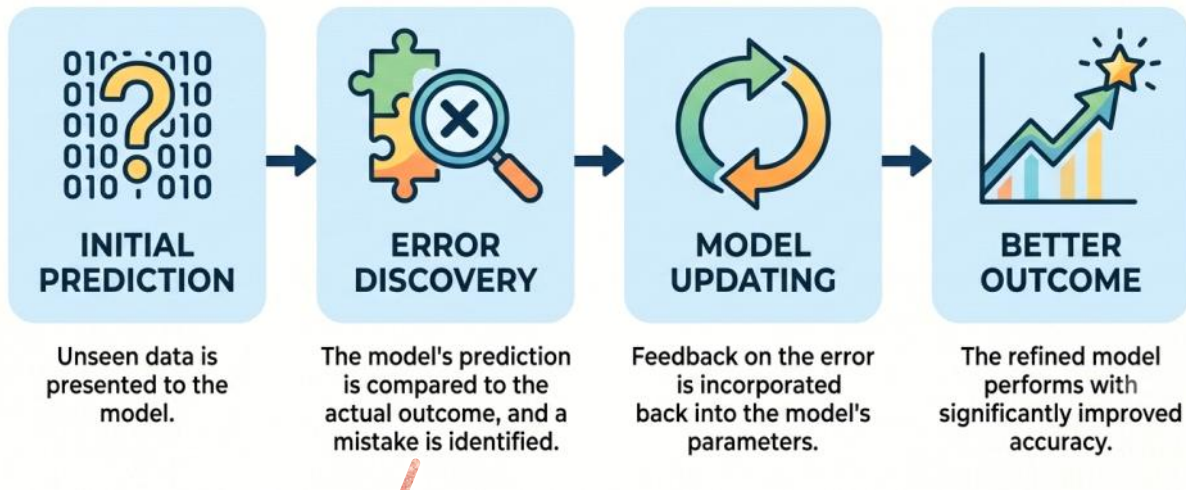
ML View - Prediction, Error and Loss

30 June 2026 20:09

A machine learning model learns only after we measure how wrong it is.

$$\frac{70}{100} \rightarrow 30\%$$

UNDERSTANDING HOW MACHINE LEARNING MODELS OPTIMIZE



One prediction gives one error.

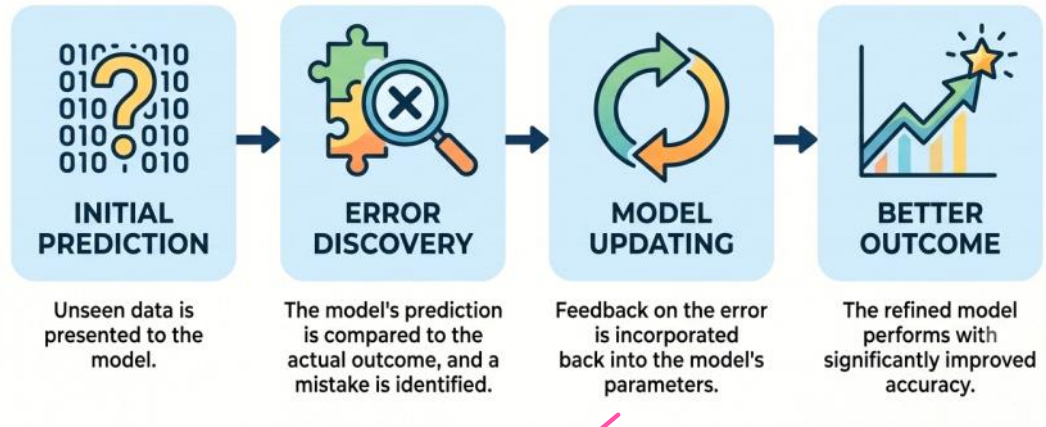
But in machine learning, we usually have many predictions.
So we need one overall number to tell us how bad the model is.

That number is called loss.

A_c	$Pr.$	E	
10	8	2	$\rightarrow error_1$
20	22	-2	$\rightarrow error_2$

Error = mistake for one prediction
Loss = overall mistake score

UNDERSTANDING HOW MACHINE LEARNING MODELS OPTIMIZE



Once the model knows the loss, it updates its internal settings.

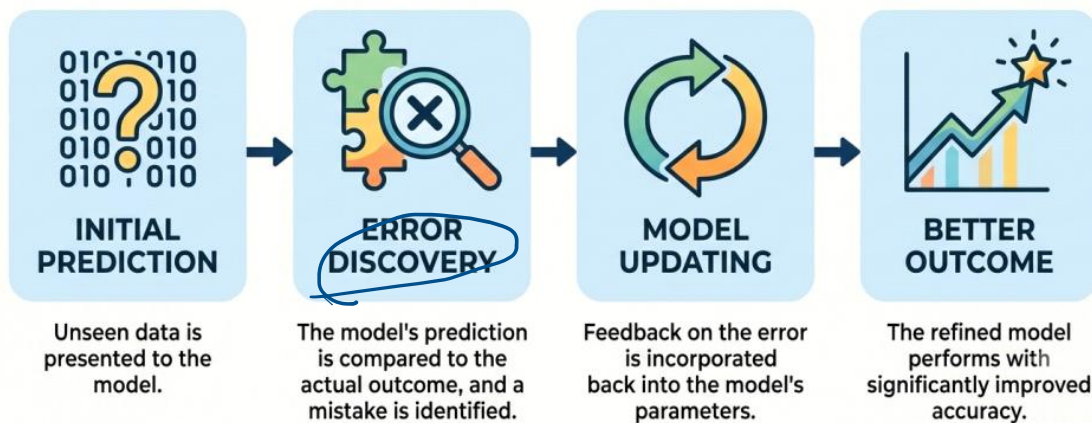
These internal settings are called **parameters**.

For example, in linear regression, the parameters are **weight and bias**.

Why? → Goal: reduce loss

How? →

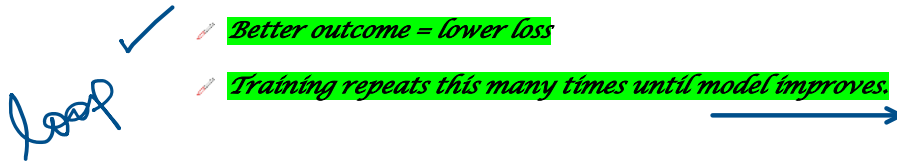
UNDERSTANDING HOW MACHINE LEARNING MODELS OPTIMIZE



After the update, the model makes a better prediction than before.

But, Better does not mean perfect.

It simply means the loss is lower than before.



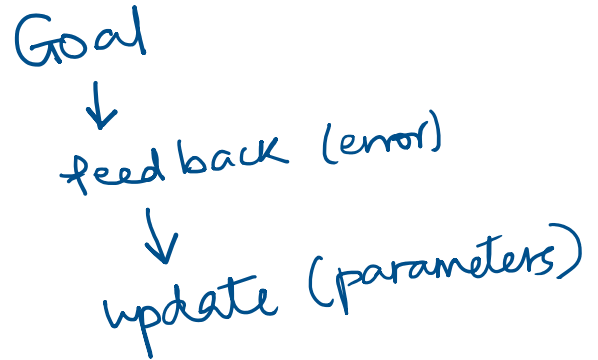
Machine learning model

Goal: Better predictions

Feedback: Prediction error

Correction: Update model parameters

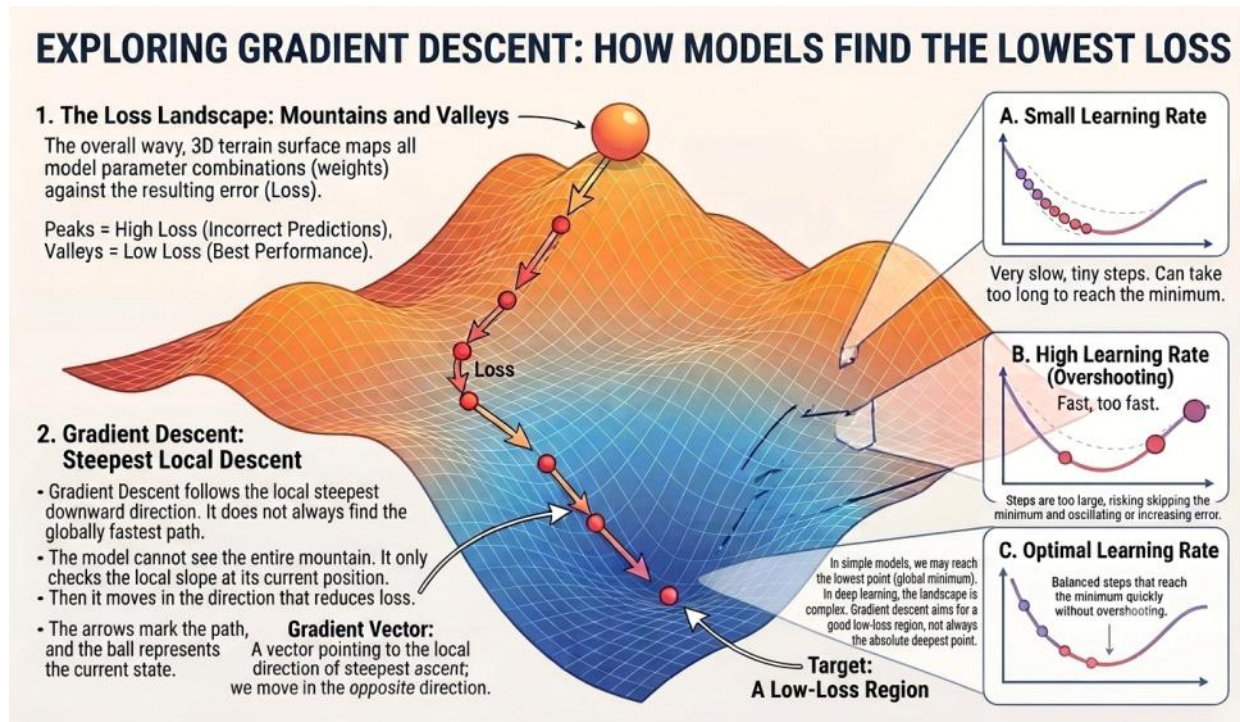
Actual	Predicted	Error
80	72	8
60	65	-5
90	85	5
50	60	-10



Loss Landscape

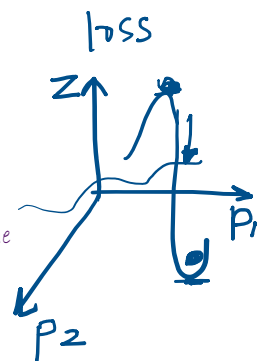
30 June 2026 20:10

When the model makes a prediction (such as identifying cats in photos or predicting house prices), the loss function tells the model exactly how far off its prediction was from the correct answer. The worse the model's prediction is, the higher the resulting loss score will be.



Imagine you are blindfolded and dropped onto a foggy mountain range. Your goal is to get down to a safe valley at the bottom.

- The Terrain (Loss Landscape):** The mountains represent your mistakes. High peaks mean you are completely wrong; the low valleys mean you are making very few mistakes and winning the game.
- The Slope:** Since you are blindfolded, you can't see the bottom. But you can use your feet to feel the ground immediately around you. If the ground tilts upward to your left, you know you need to step right to go down.
- The Gradient:** This is just another word for the **steepest direction upward** (mathematical word for slope). Since the gradient points up the mountain, **Gradient Descent** just means doing the exact opposite—stepping downward in the direction your feet tell you is the steepest slope.



Look at the image above:

Ball = current model parameters

The ball is not the data.

The ball represents the current state of the model

Downward arrows = update direction

Each arrow is one parameter update.

Gradient Descent:

Gradient points uphill.

We move opposite to gradient.

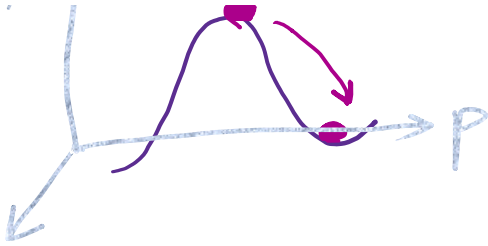
$$\text{new weight} = \text{old weight} - \text{learning rate} \times \text{gradient}$$

The Learning Rate: Your Step Size

Learning rate decides how big each update step is.

- Baby Steps (Too Small):** If you take tiny, microscopic shuffles, it will take you a year to get down the mountain. Your model learns way too slowly.
- Giant Leaps (Too Big):** If you take massive, Olympic-athlete leaps, you might accidentally jump right over the valley and crash into the opposite mountain peak. You'll just bounce around and never actually reach the bottom.





leaps, you might accidentally jump right over the valley and crash into the opposite mountain peak. You'll just bounce around and never actually reach the bottom.

- The Sweet Spot: You want a normal, steady walking pace so you can get to a good low point quickly and safely.

Putting It All Together: Training as Hill Descent

1. Start somewhere on the loss landscape with random weights.
 2. Compute the loss based on how wrong the model is.
 3. Calculate the gradient (slope) at that point.
 4. Take a step downhill, scaled by the learning rate.
 5. Repeat the process – thousands of times, across millions of data points.
- Each iteration is called a *step* or *update*. Over time, your model “descends” the terrain toward a valley – i.e., better performance.

This is the fundamental loop behind deep learning, from simple regression models to massive neural networks.

What is Gradient?

30 June 2026 20:10

loss land $\rightarrow$ mountain range

Gradient = Direction and steepness of increase.

Big idea:

Gradient tells us how loss changes when a model parameter changes.

how will we go downhill $\Rightarrow$ slope (gradient)

In simple words:

Gradient = slope of the loss curve

Gradient tells:
"How will loss change if I change this parameter?"

① Direction of change

② Steepness of change

High gradient $\rightarrow$ steep slope
Small gradient $\rightarrow$ gentle slope
Zero gradient $\rightarrow$ flat region / possible minimum



Gradient $\neq$ Gradient Descent

Gradient = slope information (Gradient is the derivative of loss with respect to a parameter)

Gradient Descent = process of using that slope to move toward lower loss

Now that we know gradient means slope and direction (First order derivative),

the next question is:

How exactly does the model use this gradient to update itself?

That process is called Gradient Descent.

Gradient Descent Update Rule

30 June 2026 20:10

Let's break down the term: **Gradient Descent** = **Gradient** + **Descent**.

- **Gradient** means the slope or steepness of a surface in a particular direction.
- **Descent** means going downward.

So, **Gradient Descent** means going downward in the direction of the steepest slope.

In simple words, gradient descent is an optimization algorithm that finds the values of weights that minimize the loss function. It does this by repeatedly adjusting the weights in the direction that reduces the loss.

1. Where am I right now?
2. Which direction increases loss?
3. Move in opposite direction
4. repeat

What is a Derivative?

A derivative tells us the rate of change of a function. In simple words, it tells us the slope of the function at a given point.

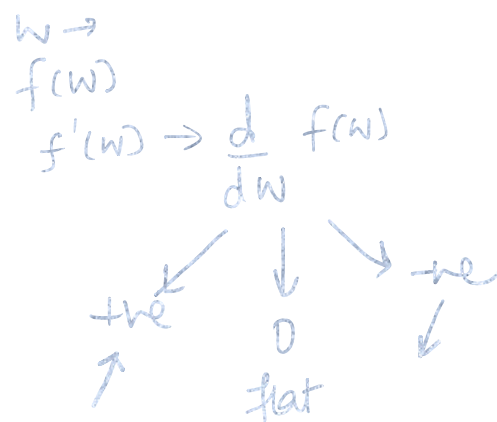
We need to know the slope of the loss function at the current weight value. The derivative tells us exactly that.

For a function $f(w)$, the derivative is written as below:

$$f'(w) = \frac{d}{dw} [f(w)]$$

Here, $f'(w)$ tells us how much the output of f changes when we change w by a tiny amount.

- If the derivative is positive, the function is going up (slope is upward).
- If the derivative is negative, the function is going down (slope is downward).
- If the derivative is zero, the function is flat (we are at a minimum or maximum).



The Update Rule

Now that we know what the derivative (gradient) tells us, we can define how to update the weight. The update rule of gradient descent is as below:

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot f'(w_{\text{old}})$$

$$w_n = w_p - \alpha \cdot f'(w_p)$$

Here:

- w_{old} is the current value of the weight
- w_{new} is the updated value of the weight
- α (alpha) is the learning rate, a small positive number that controls the step size
- $f'(w_{\text{old}})$ is the gradient (derivative) at the current weight

Now, the question is: why do we subtract the gradient?

Step-by-Step Numeric Example

The best way to understand this is by taking a concrete example with actual numbers.

Let's say our loss function is:

$$f(w) = (w - 3)^2$$

$$f(w) = (w - 3)^2$$

$$f(x) = x^2$$
$$f'(x) = 2x$$

Let's assume the model does not know this. It starts with an initial guess and uses gradient descent to find the minimum.

$$f'(w) = 2 \cdot (w - 3)$$

$$x^n \rightarrow n \cdot x^{n-1}$$

let's assume the model does not know this. It starts with an initial guess and uses gradient descent to find the minimum.

The derivative of $f(w) = (w - 3)^2$ is:

$$f'(w) = 2 \cdot (w - 3)$$

Now, let's start with $w = 0$ and a learning rate (step size) $\alpha = 0.1$, and apply the update rule step by step.

$$w_n = w_0 - \alpha \cdot f'(w)$$

Step	w (w ₀)	Gradient: 2(w - 3)	New w: w - 0.1 * gradient
1	0	2(0 - 3) = -6	0 - 0.1 * (-6) = 0.6
2	0.6	2(0.6 - 3) = -4.8	0.6 - 0.1 * (-4.8) = 1.08
3	1.08	2(1.08 - 3) = -3.84	1.08 - 0.1 * (-3.84) = 1.464
4	1.464	2(1.464 - 3) = -3.072	1.464 - 0.1 * (-3.072) = 1.7712
5	1.7712	2(1.7712 - 3) = -2.4576	1.7712 - 0.1 * (-2.4576) = 2.0170

Here, we can see that w is getting closer and closer to 3 with each step. The gradient is also getting smaller with each step, which means the steps are getting smaller as we approach the minimum. This is exactly how gradient descent converges to the minimum.

In the example above, we had only one weight w . But in a real neural network, we have millions of weights. So, how does gradient descent handle multiple weights?

Let's say we have two weights w_1 and w_2 , and our loss function is $L(w_1, w_2)$. We need to find how the loss changes with respect to each weight separately. This is called a partial derivative.

It isolates one specific weight and calculates how the loss changes with respect to that weight alone, while keeping all the other weights frozen in place.

if we have two weights, w_1 and w_2 , we calculate a unique slope for each:

- $\frac{\partial L}{\partial w_1}$ tells us the slope for weight 1.
- $\frac{\partial L}{\partial w_2}$ tells us the slope for weight 2.

$$\text{slope} = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2} \right]$$

The total gradient is simply a collection of these individual partial derivatives stacked together:

$$\text{gradient} = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2} \right]$$

Because each weight has its own dedicated partial derivative, they can be updated completely independently using the formulas:

$$w_{1_new} = w_{1_old} - \alpha \times \frac{\partial L}{\partial w_1}$$

$$w_{2_new} = w_{2_old} - \alpha \times \frac{\partial L}{\partial w_2}$$

$$w_{1_new} = w_1 - \alpha \cdot \frac{\partial L}{\partial w_1}$$

By calculating these individual slopes simultaneously, gradient descent can seamlessly scale up to manage and adjust billions of parameters at the exact same time.

$$\begin{array}{c} w_n \\ 0 \\ \downarrow \\ 0.6 \\ \downarrow \\ 1.08 \\ \downarrow \\ 1.464 \\ \downarrow \\ 1.7712 \\ \downarrow \\ 2.017 \\ \downarrow \\ 3 \end{array}$$

Learning Rate – The Most Important Knob

04 July 2026 12:00

$$f(w) = (w - 3)^2$$

The **learning rate** (α) controls how big each step is during gradient descent. Choosing the right learning rate is very important.

$$w_n = w - \alpha \cdot f'(w)$$

Let's see what happens with different learning rates using our same function

$f(w) = (w - 3)^2$ starting from $w = 0$.

Learning rate too small ($\alpha = 0.01$):

Step 1: gradient = $2(0 - 3) = -6$, new $w = 0 - 0.01 \cdot (-6) = 0.06$

Step 2: gradient = $2(0.06 - 3) = -5.88$, new $w = 0.06 - 0.01 \cdot (-5.88) = 0.1188$

Here, we can see that w is barely moving.

In fact, after 20 steps at $\alpha = 0.01$, w is only around 0.997, still nowhere near 3.

Compare this with $\alpha = 0.1$, which reached around 2.02 in just 4 steps.

Training with a tiny learning rate will be extremely slow.

Learning rate too large ($\alpha = 1.5$):

Step 1: gradient = $2(0 - 3) = -6$, new $w = 0 - 1.5 \cdot (-6) = 9$

Step 2: gradient = $2(9 - 3) = 12$, new $w = 9 - 1.5 \cdot 12 = -9$

Here, w jumped from 0 to 9 (overshooting past 3), and then from 9 to -9 (even farther away).

The value is diverging instead of converging.

This means the **learning rate is too large and gradient descent will never find the minimum.**

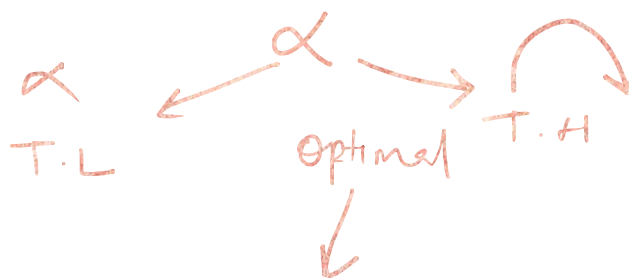
So, the learning rate must be chosen carefully.

If it is too small, training is slow.

If it is too large, training becomes unstable.

In practice, values like 0.001 or 0.01 are commonly used as a starting point.

For large models like Transformers, even smaller values like $1e-4$ are common.



$$\alpha = 0.001 / 0.01 \Rightarrow ML$$

$$\alpha = 0.0001$$

Batch GD, SGD and Mini-Batch GD

30 June 2026 20:10

Gradient Descent tells us how to update model parameters.

Batch GD, SGD and Mini-Batch GD differ in one thing: *How much data is used before one update?*

Imagine a teacher is correcting student mistakes.

Batch GD:

Teacher checks the entire class,
then gives one common feedback.

SGD:

Teacher checks one student,
then immediately gives feedback.

Mini-Batch GD:

Teacher checks a small group of students,
then gives feedback.

If we have 100 students:

Batch GD:

100 students checked → 1 update

SGD:

1 student checked → 1 update
So 100 students → 100 updates

Mini-Batch GD:

10 students checked → 1 update
So 100 students → 10 updates

Batch GD:

Stable but slower updates

SGD:

Frequent updates but noisy

Mini-Batch GD:

Balance between stability and speed

Practical Checklist + Summary

1. Optimization

- Optimization means improving something step by step using feedback.
- Optimization does **not** always mean increasing something.
- It means finding the **best value**.

2. Machine Learning as Optimization

- A machine learning model learns by improving its predictions.
- The model starts with imperfect predictions.
- It checks how wrong it is.
- Then it updates itself to make better predictions.

Core flow:

Prediction → Error → Loss → Update → Better Prediction

3. Prediction, Error and Loss

- **Prediction** is the model's guess.
- **Actual value** is the real answer.
- **Error** tells how wrong one prediction is.
$$\text{Error} = \text{Actual} - \text{Predicted}$$
- Error can be positive or negative.
- Positive and negative errors can cancel each other if we simply add them.
- So we often use squared error.
$$\text{Squared Error} = \text{Error}^2$$
- **Loss** is one overall mistake score for the model.
- In our examples, we used Mean Squared Error.
$$\text{MSE} = \text{Average of Squared Errors}$$

4. Loss Landscape

- Loss changes when model parameters change.
- Different weights and biases create different predictions.
- Different predictions create different errors.
- Different errors create different loss values.

Core flow:

Parameters change → Predictions change → Errors change → Loss changes

- A loss landscape is a visual map of how loss changes.
 - High point = high loss.
 - Low point = low loss.
 - The bottom of the valley represents a better model setting.

5. Gradient

- Gradient tells us how loss changes when a parameter changes.
- In simple terms:
Gradient = slope of the loss curve
- It tells us:
 - **Direction**
 - **Steepness**
- A large gradient means the slope is steep.
- A small gradient means the slope is gentle.
- A gradient close to zero may mean the model is near a minimum.
- Important: the gradient points toward **increasing loss**.

6. Gradient Descent

- Gradient Descent is a method used to reduce loss step by step.
- It uses the gradient to decide how to update the model.
- Since the gradient points toward increasing loss, Gradient Descent moves in the opposite direction.

Core idea:

*Gradient points uphill.
Gradient Descent moves downhill.*

Training loop:

*Start with parameters
→ Make prediction
→ Calculate loss
→ Calculate gradient
→ Update parameters
→ Repeat*

7. Gradient Descent Update Rule

The update rule is:

New weight = Old weight - Learning rate × Gradient

Or:

w_{new} = w_{old} - a × gradient

Where:

- w = weight / parameter
- a = learning rate
- gradient = slope of loss with respect to the weight

Key idea:

- *The minus sign is used because we want to move opposite to the gradient.*

8. Learning Rate

- Learning rate controls the **step size** during training.
- Gradient decides the direction.
- Learning rate decides how big the step should be.

Three cases:

- Very small learning rate:
 - Safe but slow
 - Loss reduces very slowly
- Good learning rate:
 - Smooth learning
 - Loss reduces efficiently
- Too large learning rate:
 - Steps become too big
 - Model may overshoot the minimum
 - Loss may jump or become unstable

Key line:

*Gradient = direction
Learning rate = step size*

9. Colab Example: Student Marks Prediction

We used a simple ML problem:
Study hours → Predicted marks

Model:

Predicted marks = weight × study hours + bias

Example:

bias = 40

weight is learned

The ideal weight was close to weight = 10

Because:

- 1 hour → 50 marks
- 2 hours → 60 marks
- 3 hours → 70 marks
- 4 hours → 80 marks
- 5 hours → 90 marks

10. Loss Curve in Colab

- One weight value gives one loss value.
- Many weight values give many loss values.
- Plotting weight vs loss gives a loss curve.
- The lowest point of the curve shows the best weight region.
- Gradient Descent tries to reach this low-loss region step by step.

11. Batch GD, SGD and Mini-Batch GD

All three are versions of Gradient Descent.

The difference is:

How much data is used before one update?

Batch Gradient Descent

- Uses the entire dataset before one update.
- Smooth but can be slow for large datasets.

Stochastic Gradient Descent

- Uses one data point before one update.
- Updates frequently.
- Can be fast but noisy.

Mini-Batch Gradient Descent

- Uses a small group of data points before one update.
- More stable than SGD.
- Faster than Batch GD.
- Commonly used in deep learning.

12. Important Terms

- **Optimization:** Improving using feedback
- **Prediction:** Model's guess
- **Actual value:** Real answer
- **Error:** Mistake in one prediction
- **Loss:** Overall mistake score
- **MSE:** Mean Squared Error
- **Parameter:** Adjustable value in the model
- **Weight:** A learnable parameter
- **Bias:** Constant added to prediction
- **Loss landscape:** Visual map of loss values
- **Gradient:** Slope/direction of loss change
- **Gradient Descent:** Step-by-step loss reduction method
- **Learning Rate:** Step size during update
- **Epoch:** One full pass through the training data

- **Batch GD:** Uses all data before update
- **SGD:** Uses one example before update
- **Mini-Batch GD:** Uses a small group before update
- **Overshooting:** Jumping past the minimum due to large step size
- **Convergence:** Moving toward a stable low-loss region

13. Common Mistakes to Avoid

- Error and loss are not the same.
 - Error is for one prediction.
 - Loss summarizes many errors.
- Gradient does not point toward lower loss.
 - Gradient points toward increasing loss.
 - Gradient Descent moves opposite to it.
- Learning rate does not decide direction.
 - Gradient decides direction.
 - Learning rate decides step size.
- Loss showing 0.0000 may be rounded.
 - It may not be exactly zero.
- Batch GD, SGD and Mini-Batch GD are not completely different ideas.
 - They differ only in how much data is used before each update.